

Evaluación Final: Pipeline en verde y defensa del proyecto

ASIGNATURA	PRO402 · Taller de Testing y Calidad de Software
UNIDAD	02 · Desarrollo y Ejecución de Casos de Prueba
MODALIDAD	Práctica individual · Proyecto incremental con defensa
DOCENTE	Diego Obando

1 Sentido de la evaluación

Esta es la última entrega del proyecto y cierra el arco completo del módulo. Hasta acá las pruebas existían y se ejecutaban porque alguien decidía ejecutarlas. En esta entrega dejan de depender de la voluntad de nadie: se ejecutan solas ante cada cambio, y su resultado decide si ese cambio entra o no entra.

Ese es el salto que separa un proyecto con pruebas de un proyecto confiable. Una suite que hay que acordarse de correr se deja de correr en la primera semana de apuro. Una suite conectada a la integración continua sigue trabajando cuando el equipo está distraído, que es exactamente cuando se introducen los defectos que importan.

Lo segundo que cierra esta entrega es el arco no funcional: hasta ahora todo lo probado respondía a la pregunta de si el sistema hace lo correcto. Falta la otra mitad —si lo hace lo bastante rápido, de forma segura, sin exponer datos y de manera usable— que es donde fallan los sistemas que ya funcionan bien.

Y lo tercero es que puedas sostenerlo hablando. Un informe se puede delegar; una defensa con el pipeline corriendo en vivo, no.

2 El proyecto sobre el que se trabaja

El mismo de las dos entregas anteriores, ahora completo. Lo que viene de la EP2 —el plan de pruebas, los casos derivados con técnicas formales y la suite en tres niveles— se da por

supuesto y es la base sobre la que se monta todo lo de acá.

El entorno cierra igual que empezó: `uv`, `ruff`, `pyrefly` y `pytest` como base, más FastAPI y Playwright de la entrega anterior, y la misma versión de Python de todo el módulo. Lo único que se suma es **GitHub Actions**, que no instala nada nuevo: ejecuta en un servidor exactamente los mismos comandos que corres en tu máquina. Si el pipeline necesita pasos que tu README no documenta, el entorno no era reproducible.

3

Requisitos mínimos para que la entrega sea evaluable

A. Integración continua ejecutando la suite

- Un flujo de trabajo en GitHub Actions que se dispare ante cada `push` y ante cada `pull request`.
- El flujo debe ejecutar los controles estáticos (`pyrefly`, `ruff`) y los tres niveles de la suite.
- El resultado debe ser visible: un cambio que rompe algo tiene que dejar el pipeline en rojo, y eso tiene que verse en el repositorio.
- La última ejecución sobre la rama principal debe estar en verde.

B. Pruebas de regresión y tratamiento de pruebas inestables

- Debe existir un conjunto de pruebas de regresión que proteja los defectos ya corregidos: cada defecto encontrado durante el módulo deja atrás la prueba que impide que vuelva.
- Si alguna prueba es inestable, no se esconde. Se documenta cuál es, con qué frecuencia falla, qué se investigó sobre su causa y qué decisión se tomó.
- Una prueba desactivada tiene que estar justificada por escrito. Una prueba borrada sin explicación cuenta como un defecto sin cubrir.

C. Pruebas no funcionales

Al menos **tres** categorías cubiertas con evidencia, entre las siguientes:

- **Rendimiento:** una medición sobre una operación crítica, con su umbral declarado y el resultado obtenido.
- **Seguridad:** verificación de que el sistema no expone lo que no debe y no confía en la entrada del cliente.
- **Privacidad y protección de datos:** que los datos personales que el sistema maneja tengan finalidad declarada, que se pueda ejercer un derecho sobre ellos y que exista evidencia de esa capacidad, conforme a la Ley 21.719.

- **Usabilidad o accesibilidad:** hallazgos de un recorrido exploratorio, con el problema descrito y su impacto en una persona concreta.

Cada categoría necesita un umbral o criterio declarado **antes** de medir. Una medición sin criterio previo es un número, no una prueba.

D. Plan de pruebas cerrado

`PLAN-DE-PRUEBAS.md` actualizado, con la trazabilidad completa: riesgo → requisito → caso → prueba automatizada → resultado en el pipeline. Los criterios de salida declarados en la EP2 deben aparecer contrastados con lo que efectivamente se logró, incluido lo que quedó fuera.

E. Registro del trabajo con agentes

La sección `Uso de IA o agentes` cierra con el balance del módulo completo: qué se delegó, qué se auditó, qué propuso un agente que resultó estar mal, y qué decisiones técnicas del proyecto son enteramente tuyas.

4

Entregables

1. Enlace al repositorio de GitHub, con el pipeline visible y su historial de ejecuciones.
2. `README.md` actualizado.
3. `PLAN-DE-PRUEBAS.md` cerrado.
4. `NO-FUNCIONALES.md` con las mediciones, sus umbrales declarados y los hallazgos.
5. `CALIDAD.md` y `DISEÑO-DE-CASOS.md` actualizados.
6. Código fuente, suite completa y configuración de integración continua.

5 Qué se evalúa y cuánto pesa

Criterio	Peso
Integración continua — el pipeline ejecuta lo que dice ejecutar, corre ante cada cambio y su resultado es visible y confiable	25%
Pruebas no funcionales — pertinencia de las categorías elegidas, umbrales declarados antes de medir y calidad de los hallazgos	20%
Defensa — capacidad de explicar el sistema, sus pruebas y sus límites respondiendo preguntas, con el pipeline a la vista	20%
Regresión y pruebas inestables — que los defectos corregidos queden protegidos y que la inestabilidad esté tratada y no escondida	15%
Plan de pruebas cerrado y trazabilidad — coherencia entre lo planificado, lo ejecutado y lo declarado como fuera de alcance	10%
Criterio frente al agente — el balance del módulo y las decisiones que sostienes como propias	10%

6 La defensa

La defensa es individual y se hace con el proyecto abierto y el pipeline a la vista. Consiste en responder preguntas sobre tu propia entrega. El tipo de pregunta que se hace:

- ¿Qué prueba de esta suite es la que más te costó escribir, y por qué?
- ¿Qué parte de tu sistema sigue sin estar cubierta, y qué riesgo aceptaste al dejarla así?
- Este umbral de rendimiento, ¿de dónde salió?
- Si mañana alguien cambia esta función, ¿qué se pone rojo?
- ¿Qué te dijo un agente durante el módulo que resultó estar equivocado, y cómo te diste cuenta?

No hay preguntas de memoria ni de definiciones. Todas se responden mirando tu proyecto.

Durante la defensa se abre un **pull request** con un defecto introducido en tu sistema. Se observa qué hace tu pipeline: si lo bloquea, en qué etapa, y si el resultado permite entender qué se rompió sin abrir el código.

El defecto lo genera un **agente de IA** —Claude Code o Codex, el mismo para toda la cohorte— con las mismas reglas de las entregas anteriores: sale de un catálogo fijo, es un solo cambio y

toca algo que tú declaraste. Acá no hace falta mostrarlo antes: el `pull request` queda abierto en tu repositorio y el diff está a la vista de todos, que es precisamente lo que se está evaluando.

Rige la misma impugnación de siempre: si el cambio no altera el comportamiento observable del sistema, o toca código fuera de lo que declaraste, se descarta y se genera otro.

7 Qué baja la evaluación

- Un pipeline que existe pero no ejecuta la suite completa, o que se dejó en verde desactivando lo que fallaba.
- Pruebas desactivadas, marcadas para omitir o borradas sin justificación, para llegar al verde.
- Mediciones no funcionales sin umbral declarado, o con el umbral escrito después de conocer el resultado.
- Hallazgos de usabilidad o accesibilidad genéricos, que podrían aplicarse a cualquier sistema.
- Un plan de pruebas que no coincide con lo que el pipeline ejecuta.
- Datos personales reales en el repositorio, en los fixtures, en los registros o en las capturas.
- No poder explicar durante la defensa una prueba, una configuración o una decisión de la propia entrega.
- Un pipeline que solo estuvo en verde una vez, sin historial que muestre el trabajo.

8 Política de IA y agentes

Rige lo mismo que en las entregas anteriores. Lo que cambia es que la defensa hace visible de inmediato la diferencia entre haber usado un agente y haber sido usado por él.

Todo lo que esté en tu repositorio tienes que poder explicarlo: qué hace, por qué está ahí y qué pasaría si se quitara. Eso incluye el archivo de configuración del pipeline.

9 Método de entrega

1. Sube el proyecto a GitHub, en el mismo repositorio de las entregas anteriores.
2. Verifica que sea público o esté compartido con el docente, y que las ejecuciones del pipeline sean visibles.
3. Confirma que el `README.md` permite instalar, ejecutar y probar el proyecto completo.

4. Envía el enlace por el canal oficial de la asignatura.

La defensa se realiza sobre el estado que tenga el repositorio en ese momento.